

ACTIVIDAD 3-FUNCIONES

Instrucciones

1. A partir de la actividad anterior vamos a crear un **formulario de login que incluya un captcha** aritmético. El resultado final deberá se similar al siguiente:

Ver imagen JS-12.gif

Usuario:

Contraseña:

7 + 8 =

Comprobar

Submit

Se proporciona el código HTML inicial en el siguiente [enlace](#) .

Para realizar la actividad necesitamos conocer previamente un par de cosas sobre cómo ejecutar código cuando se pulsa un botón y como modificar dinámicamente una página HTML:

- En el **HTML**
 - Si queremos ejecutar código cuando suceda un evento, una posibilidad es asociar una llamada a una función en la propiedad asociada a evento correspondiente (onclick, onmousedown, onkeypress...). Por ejemplo:
`onclick="funcion(parámetros)"`
 - A la hora de especificar estas funciones podemos **acceder a los controles de los formularios** a través de su propiedad `id`. Por ejemplo, si tenemos un **control de texto** llamado `usuario`, podemos acceder a lo que contiene con `usuario.value`. Por tanto, si al hacer click en un botón queremos llamar a una función a la que pasamos el nombre que el usuario ha escrito pondremos `onclick="funcion(nombre.value)"`
- En el código **Javascript**
 - **Si queremos acceder** a cualquier elemento (`DIV`, `SPAN`, `BODY`, `INPUT...`) de nuestra página HTML una posibilidad es asignarle un `id` y emplear en Javascript el método `document.getElementById("id")`
 - **Una vez accedido** al elemento
 - **Si es un elemento estructural** (`DIV`, `SPAN`, `H1`, `P...`) podemos modificar su contenido con la propiedad `innerHTML` que sustituirá el contenido actual del elemento por el especificado

- **Si es un control** podemos acceder (modificar) a su contenido con la propiedad `value`.

Ejemplo

[Ver código en JSFiddle](#)

Como se puede ver podemos **cambiar dinámicamente el contenido** de la página HTML accediendo al elemento (`document.getElementById`) y especificando su nuevo contenido en la propiedad `innerHTML`.

Pasos

- Al hacer `click` en el botón `enviar` habrá que llamar a la función `validar` a la que se pasarán como **parámetros** el contenido de los controles `usuario` y `pwd` de manera similar a como se ve en el ejemplo anterior
- Por otro lado, al hacer `click` en el botón `Comprobar` se llamará a un método `comprobar` **sin parámetros**.
- Todo el código irá en un fichero javascript externo.
- En dicho fichero
 - Se **declararán tres variables** para el captcha (`operando1`, `operando2` y `operador`) mediante la instrucción `let`.
 - Se definirá una variable (`captcha`) a la que se asignará una expresión de función que:
 - Obtendrá **valores aleatorios** entre 1 y 10 (sin incluir) para `operando1` y `operando2`
 - Obtendrá la **operación** a realizar (`+`, `-`, `*`) de manera **aleatoria**
 - **Asignará** esos valores (con `innerHTML`) a los `span` correspondientes
 - Tras ello se **llamará a la función anterior**
 - **COMPROBACIÓN:** Si ejecutamos la página debería aparecer un captcha diferente cada vez que recargamos la página
 - Tras ello definiremos el método `comprobar` con una función normal. En dicho método:
 - **Obtendremos el resultado escrito** por el usuario en el cuadro de texto `rd` (`document.getElementById` y `value`)
 - **Calcularemos el resultado esperado** en base a `operando1`, `operando2` y `operador` (podéis investigar la función `eval` de Javascript)

- Si **ambos coinciden**
 - Pondremos la propiedad disabled del control enviar a false
 - Eliminaremos el captcha poniendo su innerHTML en **blanco**
- Si no
 - Dejaremos el cuadro de texto rdo (su valor) en **blanco**
 - Llamaremos a la función captcha para generar un nuevo reto
- **COMPROBACIÓN:** Ahora el botón Comprobar ya debería funcionar correctamente. Si el usuario acierta, se activa el botón y desaparece el captcha, si no se genera otro automáticamente
- Por último definiremos la función validar mediante una **función de flecha** (recordar que recibe dos parámetros). En esta función:
 - Si el usuario / pwd que recibe **coincide** con uno que vosotros **especifiquéis**:
 - Se **muestra una alerta**
 - Se **sustituye** todo el contenido por una **bienvenida** (h1) al usuario
 - Si no
 - Se muestra un mensaje de **error**
- **COMPROBACIÓN:** La aplicación ya debería funcionar como en el ejemplo.

2.

En este caso vamos a hacer un ejemplo de funciones callback (funciones a las que se pasa como parámetro otra función). Los pasos a dar serán los siguientes:

- Definir una **función de expresión** almacenada en una variable llamada **negrita** que reciba un texto y retorne el HTML necesario para poner **dicho texto en negrita**
- Definir una **función de flecha** almacenada en una variable llamada **cursiva** que reciba un texto y retorne el HTML necesario para poner **dicho texto en cursiva**
- Definir una función llamada textoF que reciba un texto y un **parámetro que será una función** (este parámetro será opcional). La función textoF
 - si se ha recibido una función, retornará un div con el resultado de **llamar a la función recibida** como parámetro con el texto
 - si no, retornará un div con el texto
- **Probar en la consola:**
 - textoF("Hola",cursiva) → "<div>Hola</div>"
 - textoF("Hola",negrita) → "<div>Hola</div>"
 - textoF(textoF("Hola",cursiva),negrita) → "<div><div>Hola</div></div>"

- `textoF("Quién eres?",(t) => prompt(t))` → Intentar adivinar qué va a pasar antes de probarlo
- Con el código hacer una minipágina que permita activar esas características sobre un cuadro de texto:

Ver imagen JS-13.gif

☐ Negrita ☐ Cursiva

